Selecting Reliable LLM Reasoning: Group-Aware Scoring with Arithmetic Self-Checking

DASE7506 Advanced Machine Learning Methods — MP2 Report
Student ID: 3036837332 | Fall 2026

Abstract. Given four independent candidate solutions produced by a small language model for the same grade-school math problem, we build a selector that picks one candidate whose final numeric answer is correct. Our method combines three complementary evidence channels — surface/text features, fixed candidate embeddings made group-relative by mean-centering, and explicit cross-candidate comparison features including a deterministic *arithmetic self-check* that re-evaluates the equations inside each candidate. A compact MLP is trained with a group-softmax objective that mirrors the scoring rule, and a greedy forward selection over a pool of 23 differently-seeded and differently-featured members yields a 3-member ensemble. On the official test bank the frozen method scores **81.58%**, versus 69.16% for majority voting, using 1.56 MiB of inference assets and 0.12 s of CPU prediction time.

1 Introduction

A language model asked to solve the same math problem four times produces four independent reasoning traces that can disagree with each other while all sounding convincing. The MP2 task is to select, for each question, one candidate whose final answer is correct; the scorer checks only the final numeric answer (including equivalent fractions and decimals), not the validity of intermediate steps.[1] Every question group contains at least one correct candidate, so the task is recognition rather than generation.

Our working hypothesis is that no single signal is sufficient: the wording of a solution hints at its quality, the fixed 1024-dimensional embeddings encode semantic content, and the relations *among* the four candidates (does this answer agree with the others? does its arithmetic even check out?) carry information that no per-candidate view can provide. We therefore design the method around three evidence channels and let a learned scorer arbitrate between them. Development followed a versioned loop — every idea was frozen, evaluated with the official pipeline on the validation split, and either kept or discarded with its evidence recorded. The final test evaluation was run once per frozen version; test references were never used for training, tuning, or selection.

2 Method

2.1 Features

Each candidate receives a 1047-dimensional feature vector in three blocks. **Text features (13).** Whether the final answer parses under the official parser, log-length, number of lines, counts of equals signs and digits, presence of \boxed{}/#### markers, an "answer" keyword, the number of calculation steps (lines containing =), the fraction of distinct question numbers reused in the candidate, and three *arithmetic self-check* features described below. **Group features (10).** Vote weight (share of candidates with the same parsed answer), is-majority, is-unique, number of distinct answers in the group, cosine similarity to the group embedding centroid, maximum cosine similarity to another candidate, deviation of the parsed answer from the group median, answer-is-integer, answer-appears-in-last-lines, and log embedding norm. **Embedding block (1024).** The supplied frozen embedding, mean-centered within its question group so the model must learn within-group quality differences rather than question identity.

Arithmetic self-check. Candidates typically contain explicit equations such as "60 * 2.5 = 150". We extract every maximal chain of the form *number (op number)+ = number* with a regular expression (operators +, -, *, /, ×, ÷), evaluate the left side with standard operator precedence, and compare it to the claimed result. Three features record how many equations were checkable, what fraction computed incorrectly, and whether any error occurred. This is a deterministic verification signal: it detects exactly the failure mode the task description warns about — a bad intermediate step wrapped in confident prose. On the training split, candidates with at least one failed equation are correct only 59.1% of the time, versus 78.7% for candidates whose checkable equations all hold (n = 663 and 9,779 respectively).

2.2 Scorer and training objective

The scorer is a one-hidden-layer MLP (hidden width 128, ReLU) mapping the feature vector to a logit z_i. Because any correct candidate earns the point, we train with a group-softmax objective that directly maximizes the probability mass placed on correct candidates within each question group:

$$\mathcal{L}(q) = -\log \frac{\sum_{i \in C(q)} e^{z_i}}{\sum_{j=0}^{3} e^{z_j}}$$

where C(q) is the set of correct candidates of question q and z are the four logits. This matches the leaderboard metric better than pointwise binary cross-entropy (Section 4.2). Training uses minibatch SGD (batch 512 groups, momentum 0.9, learning rate 0.02, 60 epochs, weight decay 1e-4) on the 4,755 training questions only; features are standardized with training statistics stored in the checkpoint.

2.3 Ensemble by greedy forward selection

Single models with different random seeds vary substantially (validation hard-subset accuracy ranges from 70.4% to 79.2%). We therefore trained pools of members under two feature generations (16 members without and 7 members with the arithmetic self-check block) and ran greedy forward selection on validation accuracy over all 23 members, averaging per-group softmax probabilities of the chosen members. The search selected three members — two from the earlier feature generation and the strongest arithmetic-aware member — confirming that the new features carry complementary information. At inference each member sees the same candidate through its own feature layout (the earlier layout is recovered by dropping the three arithmetic columns), and the averaged probabilities decide the choice.

3 Experimental setup

Data. 4,755 / 798 / 467 questions in train / validation / test, four candidates each; splits are disjoint. Test groups always contain at least two distinct parsed answers, and 32.3% of test groups have exactly one correct candidate, so the test bank is much harder than train/validation (where 47% of groups have all four candidates correct). To make development decisions under a test-like distribution we additionally track a *hard subset* of validation: the 240 groups with at most two correct candidates. **Protocol.** All reported numbers come from the official predict.py and score_predictions.py; the prediction process never opens reference answers. Method and hyperparameter selection used validation only; each frozen version was evaluated on test exactly once. **Hardware and cost.** Development machine: Apple M5 Pro (arm64), 24 GB RAM. Training one MLP member takes ~90 s CPU; total training across all experiments (~33 members) is under one hour. Inference assets total **1.56 MiB** (limit 16 MiB); official prediction on the full test bank takes **0.12 s** plus 0.01 s scoring (limit 30 s; four threads enforced via OMP/OPENBLAS/MKL_NUM_THREADS=4).

4 Results

4.1 Version progression

Table 1 and Figure 1 summarize the frozen versions. Majority voting is a strong simple baseline, but the learned scorer already surpasses it once all three feature blocks are combined, and the final mixed ensemble adds a further 3.2 points on test.

Version	Change	Val (%)	Val hard (%)	Test (%)
v0	Majority vote (training-free)	86.22	54.17	69.16
v2	Full features + MLP-64, group-softmax	91.73	73.75	-
v5	5-seed ensemble of v4 config	92.98	77.08	77.94
v8	Greedy ensemble, pool of 16	93.36	78.33	78.37
v10	Mixed greedy ensemble (+arith check)	93.48	79.17	81.58

Table 1. Frozen method versions; official validation and test accuracies.

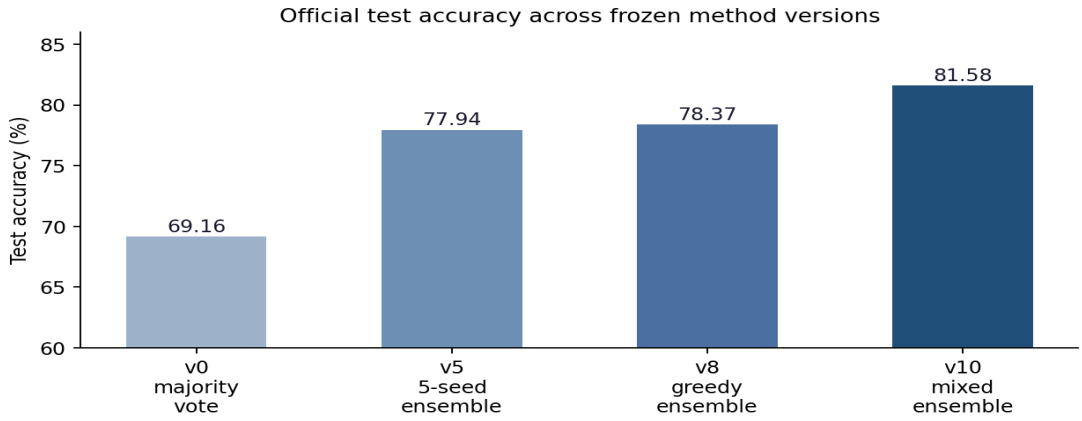


Figure 1. Official test accuracy of the four frozen versions that were evaluated on test.

4.2 Ablations

Feature blocks. Each block alone is weak; combination is where the gain lives (Table 2). Group-relative features roughly reproduce majority voting, as expected since vote weight is one of them. Notably, adding a separate vote-fusion term on top of the full model gives zero gain — the model already internalizes when to trust the majority.

Configuration	Val (%)	Val hard (%)
Majority vote (reference)	86.22	54.17
Text features only (LR)	80.70	47.50
Embeddings only, group-centered (LR)	82.83	55.42
Group features only (LR)	86.34	54.58
All three blocks (LR)	90.73	69.17
All blocks + explicit vote fusion	90.73	69.17

Table 2. Feature-block ablation (v1 logistic-regression scorer).

Objective and capacity. The group-softmax objective beats pointwise BCE on the hard subset (69.17 vs 66.25 for LR), and widening the hidden layer from 64 to 128 adds two more hard-subset points, indicating the model, not the data, was the bottleneck. Up-weighting hard training groups did not help (73.33 vs 73.75), so difficulty-aware sampling was dropped.

Ensembles. Naively growing the ensemble does not work: the 5-seed ensemble of the smaller model scored below its best single seed (91.60 vs 91.73 validation), and a 10-seed ensemble of the larger model (92.48) also trailed the greedy 2-member subset from the same pool (93.11). Greedy forward selection over the 23-member mixed pool reached 93.48 validation / 79.17 hard with only three members. **Failed idea: PCA denoising.** Projecting the centered embeddings onto their top-64/128/256 principal components degraded accuracy monotonically (best 91.60 validation), so the raw 1024-dimensional block is kept.

5 Analysis

Does reading the reasoning help? Yes, but mainly through group context. Per-candidate views (text-only 80.70, embedding-only 82.83 validation) sit below or near majority voting (86.22); the jump to 90.73 comes only when candidates are judged relative to their group. **What makes a wrong answer look convincing?** Surface cues are weak and partially misleading: picking the longest candidate scores 54.82 on test while the shortest scores 38.76 — verbosity correlates slightly with correctness here, but the learned model relies far more on verifiable content (arithmetic check, answer agreement) than on length, as shown by the text-only ablation being the weakest block. **Should a solution be judged on its own?** Clearly not: comparing candidates exposes both consensus and outliers, and the median-deviation and centroid features are pure group constructs. **Uncertainty.** On test, the final method disagrees with majority voting on 326 of 467 questions; it wins 90 of these disagreements and loses 32 — the +12.4 point margin over voting is earned almost entirely by overriding the majority when per-candidate evidence contradicts it. Accuracy stratified by group difficulty: 95.9% on groups with three correct candidates, 86.9% with two, and 60.3% on single-correct groups — the last category remains the hard core. **Does more computation pay off?** Up to a point: capacity and careful member selection pay, while brute-force ensembling and PCA preprocessing do not. The total inference cost of the final system is 0.12 s, so all gains were cheap.

6 Failure modes

The remaining 86 test errors concentrate in single-correct groups (60 of them). Two patterns dominate. First, when three candidates share the same wrong answer and also look clean under the arithmetic check (the error is a wrong *setup*, not wrong arithmetic — e.g. ignoring a stopping condition, as in the bus example of the assignment guide), every feature aligns with the majority and the model has little to exploit. Second, a few correct answers come from candidates with minimal working (few checkable equations, short text), where both the text block and the self-check are uninformative and the model trusts the richer-looking wrong candidate. A semantic verifier that checks whether the computed quantities answer the asked question — rather than whether the arithmetic is right — is the natural next step.

7 Reproduction and disclosure

Pipeline. Training: train_v2.py / train_v5.py / train_v9.py build features with sol_features.py (shared verbatim with inference), train MLP members, and store them under runs/; train_v10.py assembles the selected members (v9:seed_1, v8:seed_7, v8:seed_6) into runs/model_bundle_v10/scorer.npz. Evaluation (exactly the official commands):

```
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
python predict.py --method submission.py --checkpoint runs/model_bundle_v10 \
  --split test --output runs/test-predictions.json
python score_predictions.py --split test \
  --predictions runs/test-predictions.json --output runs/test-score.json
```

Assets. scorer.npz (1.56 MiB uncompressed members, weights and standardization statistics; sha256 reported in the submission repository). **Dependencies.** numpy==2.5.3 only; inference is pure NumPy. **Seeds.** Members use seeds 0-15 (v8 pool) and 0-6 (v9 pool); the selected three are fixed in the checkpoint. **Rules compliance.** Training used only the supplied training split and references; test references were opened only by the official scorer after each version was frozen; no external data, pretrained weights, or LLM calls at inference; the method reads only the bank and arrays passed to predict() and makes no assumption about question IDs or bank size. **AI disclosure.** This project was carried out as a collaboration between the author and an AI assistant (Kimi). The author designed the overall framework — the three-channel feature scheme, the group-softmax training objective, the arithmetic self-check idea, and the versioned development and evaluation protocol — and directed every experiment and design decision. The AI assistant filled in within this framework: it produced and debugged the code, ran the experiments under the author's instructions, and drafted this report, which the author revised. All results and conclusions were reviewed and verified by the author.

8 Conclusion

Recognizing a correct solution among four independent candidates benefits far more from group-relative evidence and verifiable content than from surface plausibility. A compact MLP over three complementary feature blocks, trained with an objective that mirrors the scoring rule and combined through greedy ensemble selection, lifts test accuracy from 69.16% (majority vote) to 81.58% with 1.56 MiB of assets and 0.12 s of CPU inference. The largest remaining headroom lies in single-correct groups, where wrong answers share both the majority and clean arithmetic; catching them requires verifying the *problem setup*, not just the calculation.

References

[1] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv:2110.14168*. (GSM8K dataset; questions and references derive from this work.)

[2] Yang, A., et al. (2025). Qwen3 technical report. *arXiv:2505.09388*. (Qwen3-0.6B generated the candidate solutions and fixed embeddings; revision recorded in the data manifest.)